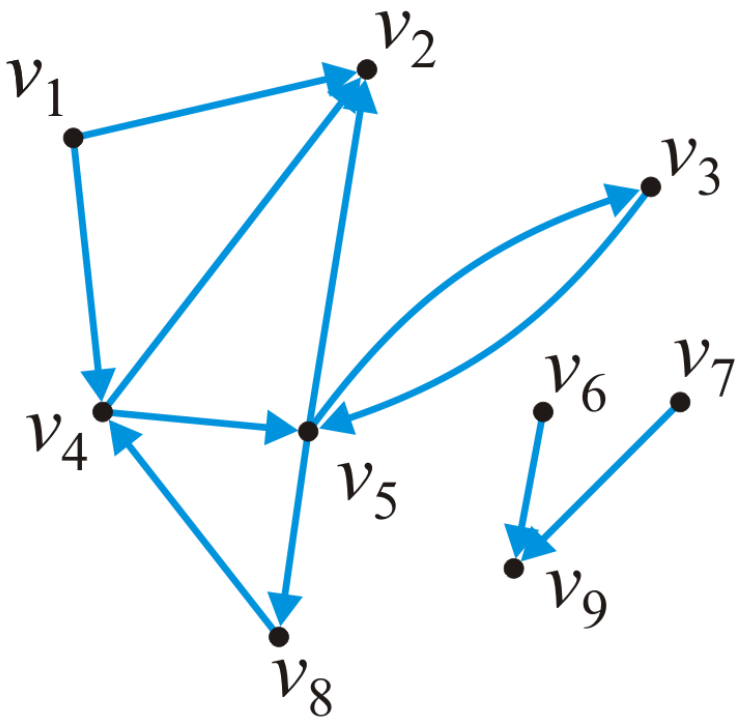
Data Structures

15. Graphs

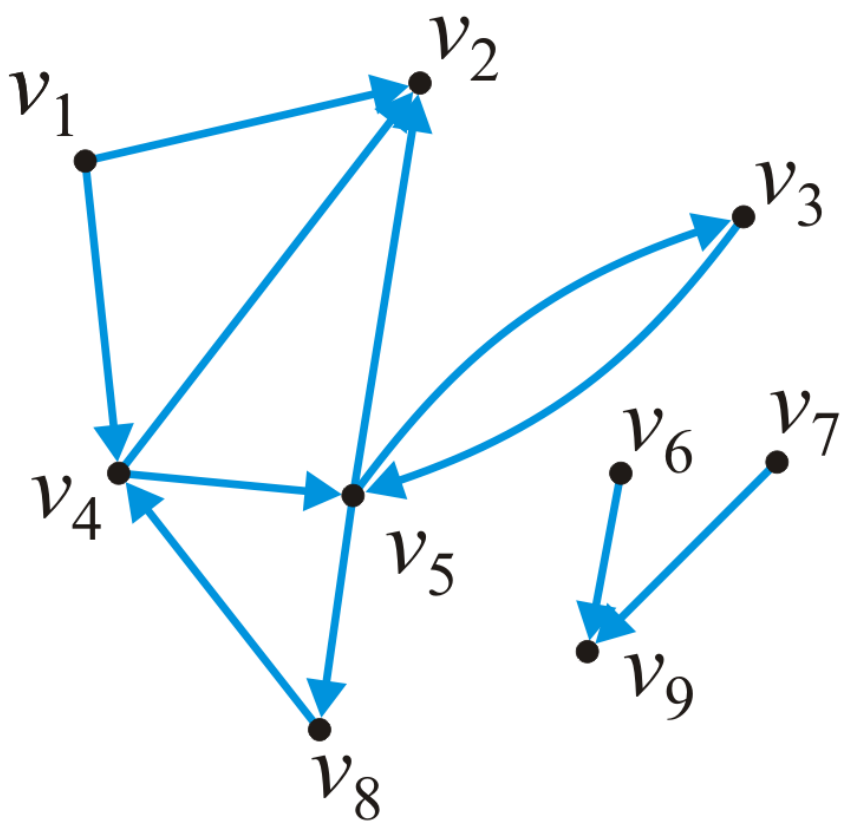
Connectedness

- Two vertices v_j, v_k are said to be connected if there exists a path from v_j to v_k
  - A graph is strongly connected if there exists a directed path between any two vertices
  - A graph is weakly connected if there exists a path between any two vertices that ignores the direction



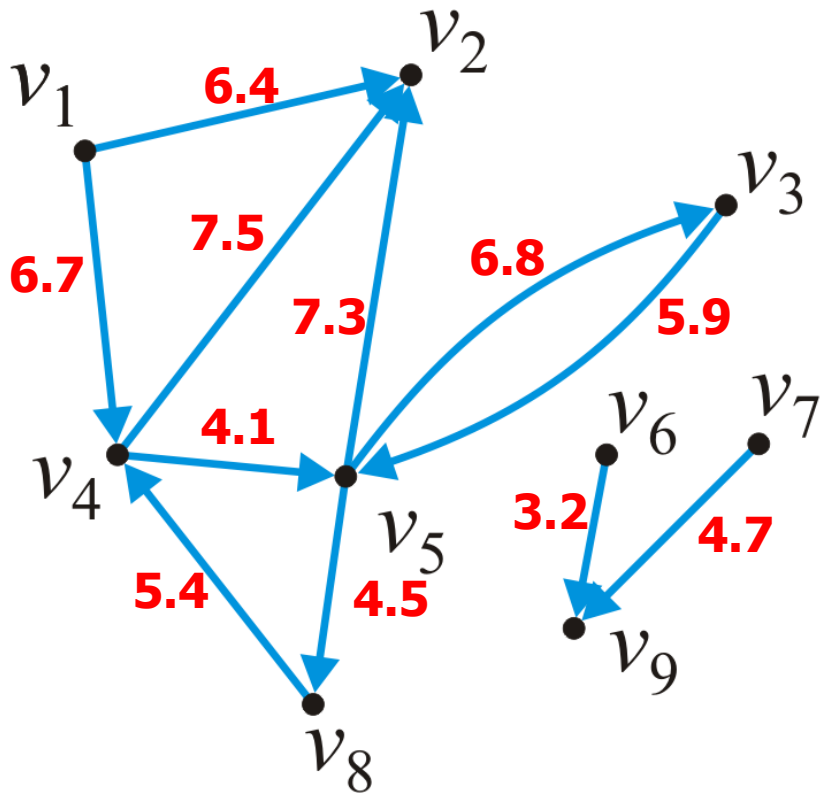
Connectedness – Example

- The sub-graph $\{v_3, v_4, v_5, v_8\}$ is strongly connected
- The sub-graph $\{v_1, v_2, v_3, v_4, v_5, v_8\}$ is weakly connected



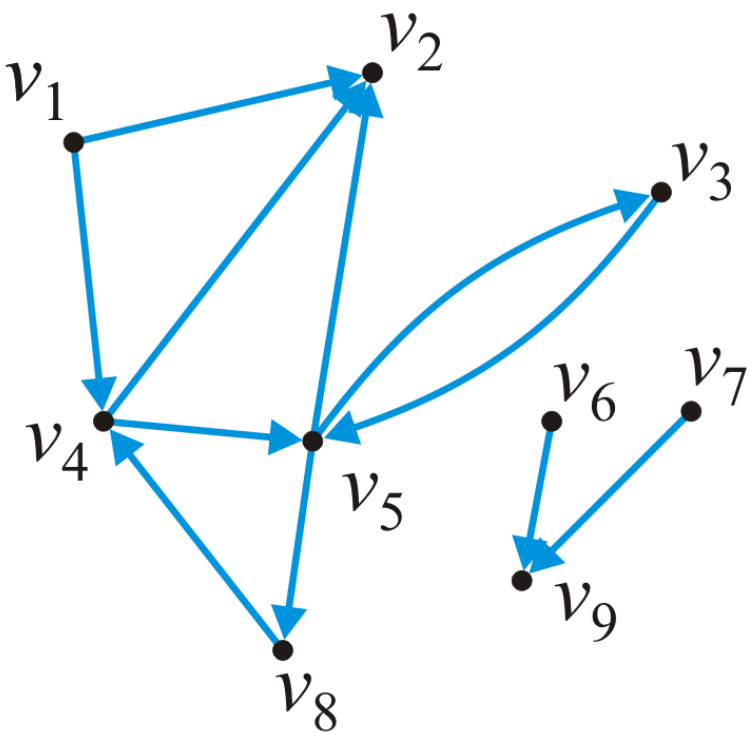
Weighted Directed Graphs

- Each edge is associated with a value
- If both (v_j, v_k) and (v_j, v_k) are edges
  - It is not required that they have the same weight



Representation

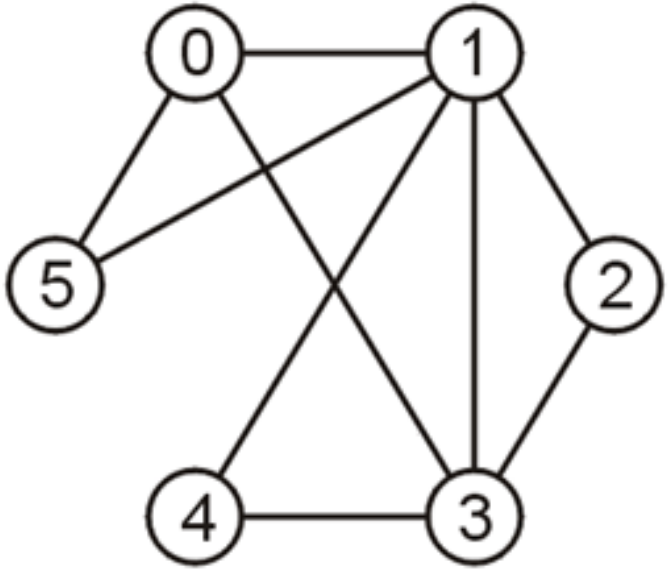
- How do we store the adjacency relations?
  - Adjacency matrix
  - Adjacency list



Adjacency Matrix

- Two dimensional matrix of size `n x n` where `n = |V|`
- `a[i, j] = 0 (F)` if there is no edge between vertices v_i and v_j
- `a[i, j] = 1 (T)` if there is an edge between vertices v_i and v_j
- Adjacency matrix of undirected graphs is symmetric
  - `a[i, j] = a[j, i]`

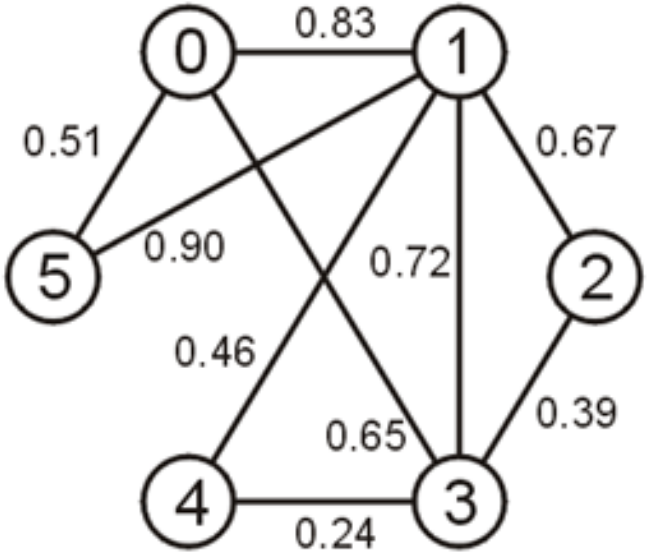
	0	1	2	3	4	5
0	F	T	F	T	F	T
1	T	F	T	T	T	T
2	F	T	F	T	F	F
3	T	T	T	F	T	F
4	F	T	F	T	F	F
5	T	T	F	F	F	F



Adjacency Matrix – Weighted Graph

- The matrix entry `[j, k]` is set to the weight of the edge (v_j, v_k)
- How to indicate absence of an edge in the graph?

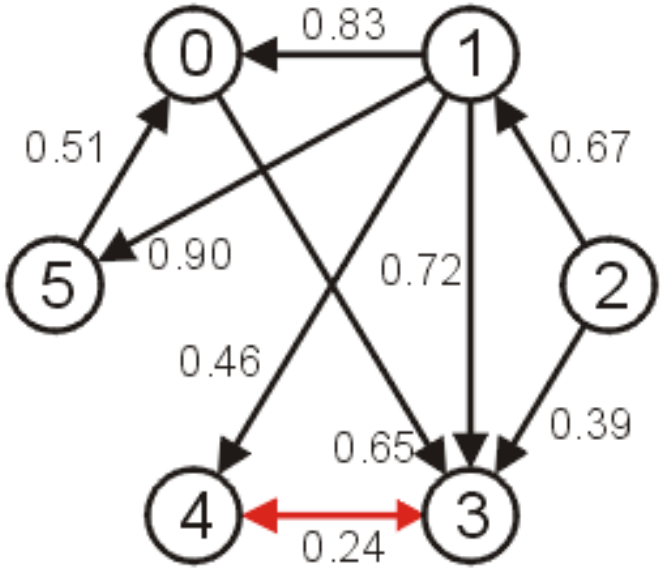
	0	1	2	3	4	5
0		0.83		0.65		0.51
1	0.83		0.67	0.72	0.46	0.90
2		0.67		0.39		
3	0.65	0.72	0.39		0.24	
4		0.46		0.24		
5	0.51	0.90				



Adjacency Matrix – Directed Graph

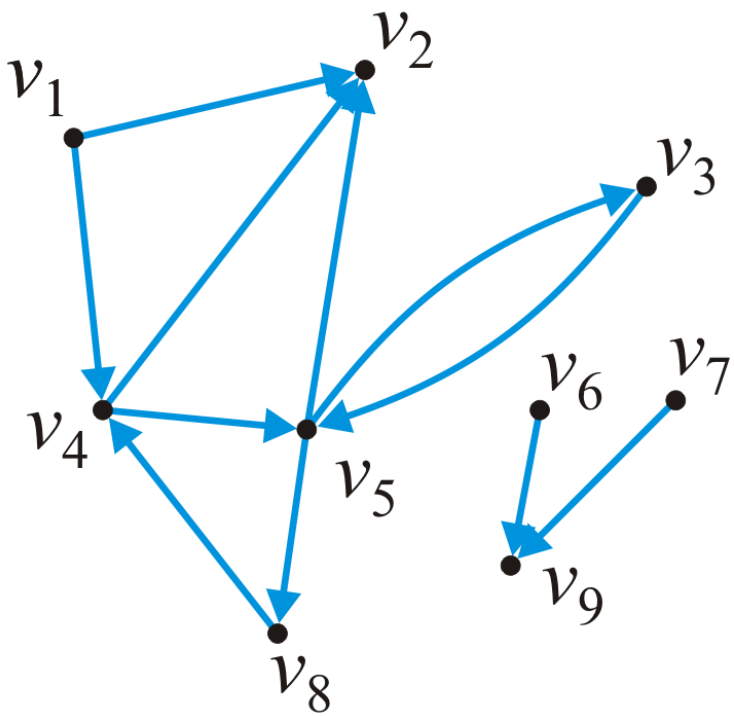
- For directed graph the matrix would not necessarily be symmetric

	0	1	2	3	4	5
0				0.65		
1	0.83			0.72	0.46	0.90
2		0.67		0.39		
3					0.24	
4				0.24		
5	0.51					



Adjacency Matrix – Analysis

	1	2	3	4	5	6	7	8	9
1		1		1					
2									
3					1				
4		1			1				
5		1	1					1	
6									1
7									1
8				1					
9									

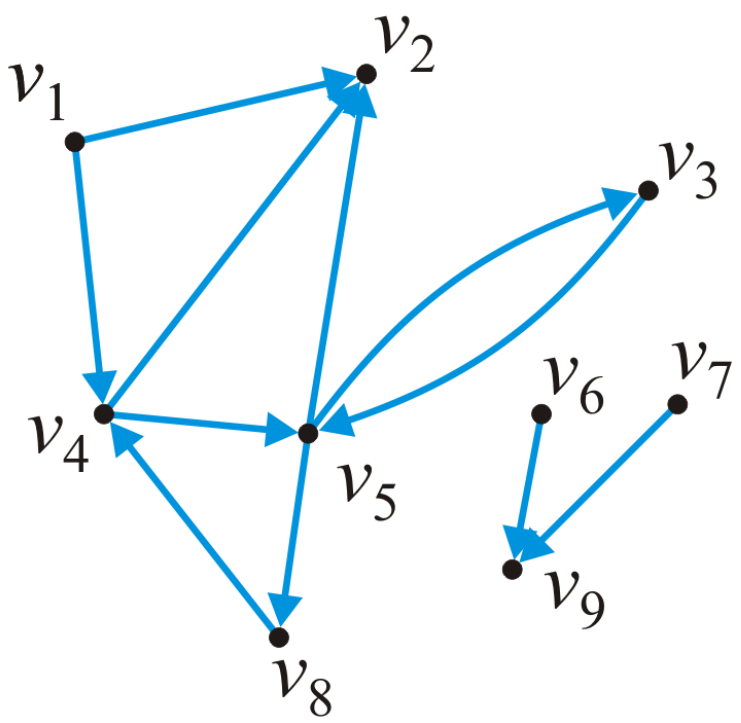


- Requires memory : $O(|V|^2)$
- Determining if v_j is adjacent to v_k : $O(1)$
- Finding all neighbors of v_j : $O(|V|)$

Adjacency Matrix – Problem

- Very sparsely populated
  - Out of 81 cells only 11 are 1 (or T)

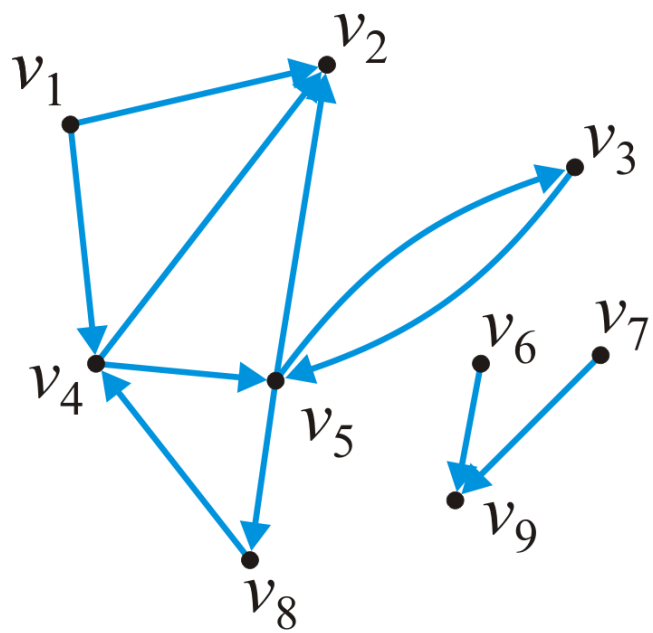
	1	2	3	4	5	6	7	8	9
1		1		1					
2									
3					1				
4		1			1				
5		1	1					1	
6									1
7									1
8				1					
9									



Adjacency List

- Each vertex is associated with a list of its neighbors
  - A vertex `w` is inserted in the list for vertex `v` if edge `(v, w)` exists

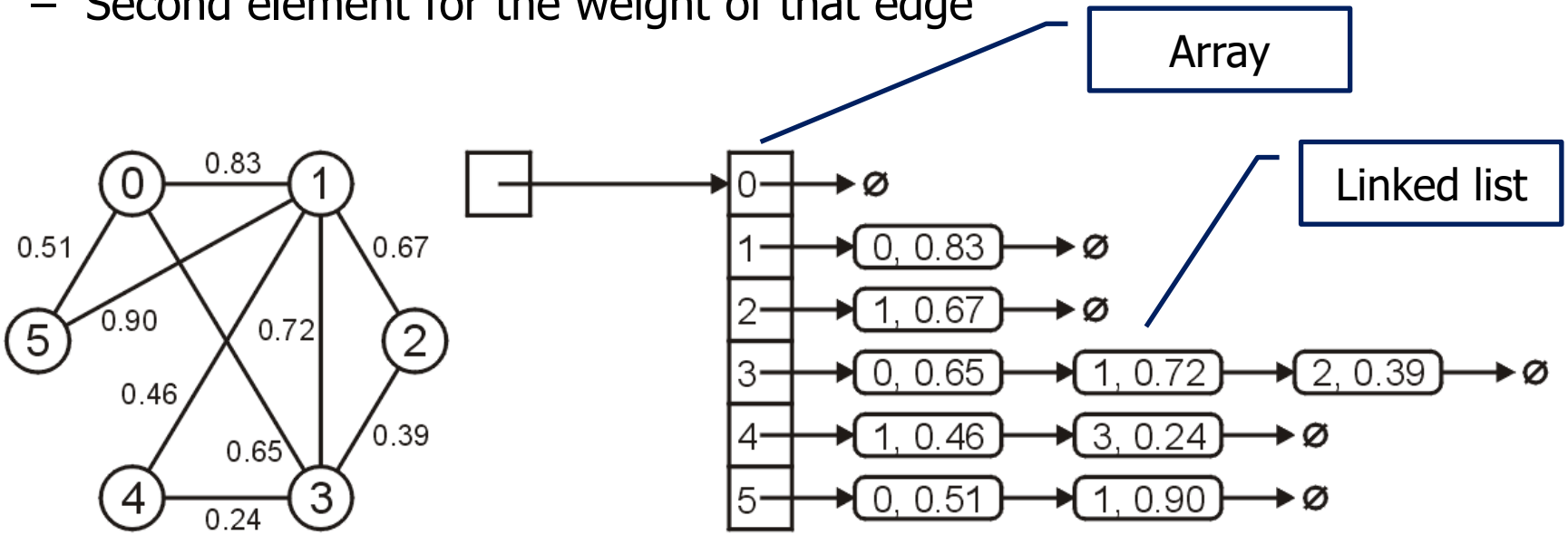
1	• → 2 → 4
2	•
3	• → 5
4	• → 2 → 5
5	• → 2 → 3 → 8
6	• → 9
7	• → 9
8	• → 4
9	•



- Requires memory : `O(|V| + |E|)`

Adjacency List – Weighted Graphs

- An adjacency list for a weighted graph contains two elements
  - First element for the vertex
  - Second element for the weight of that edge

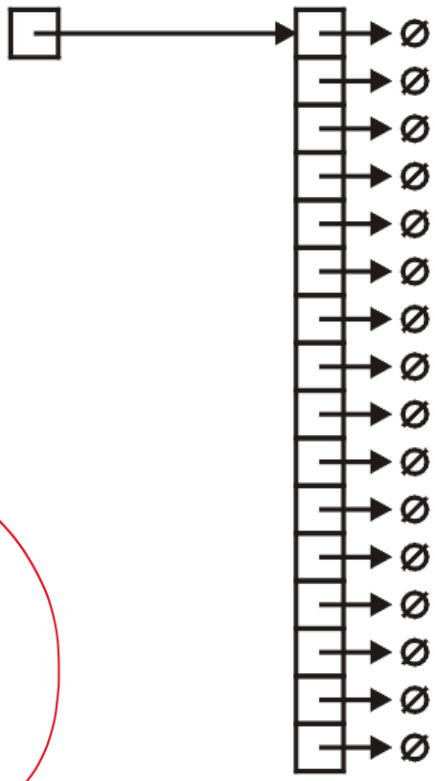


- When the vertices are identified by a name (i.e., string)
  - Hash-table of lists is used to implement the adjacency list

Adjacency List – Implementation

- Node to store adjacent vertex and weight of the edge

```
class SingleNode {
   private:
      int adjacent_vertex;
      double edge_weight;
      SingleNode * next_node;
};
```



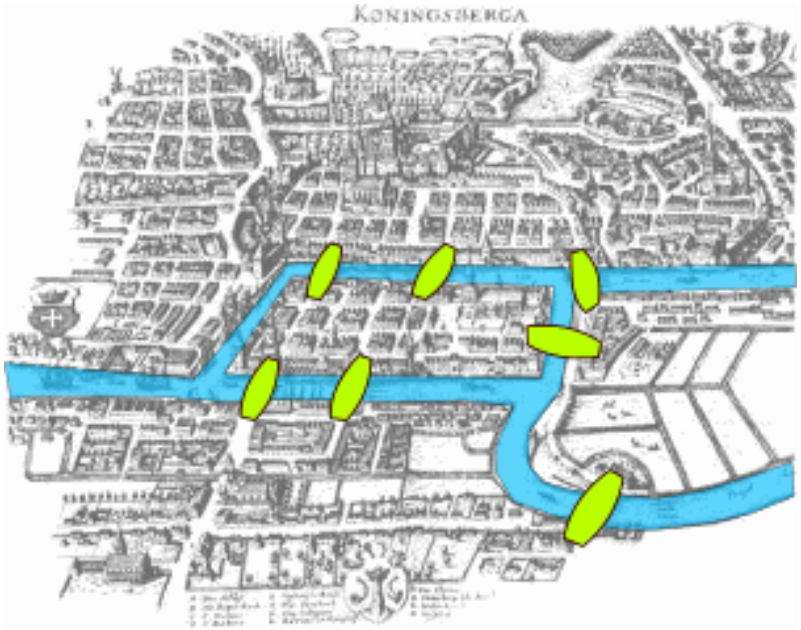
- Define and create Array

```
SingleNode * array;
array = new SingleNode[16];
```

Graph Problems – Euler Tour

- A sequence of vertices that traverse all edges in the graph exactly once
  - Leonhard Euler in 1736
  - Laid the foundations of graph theory

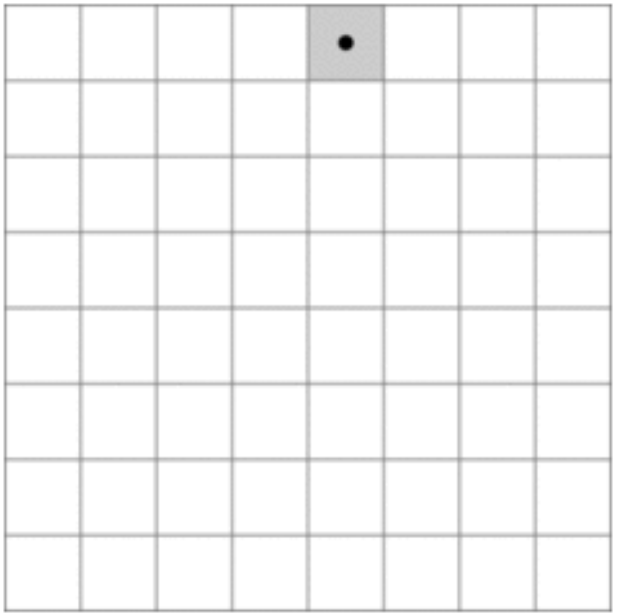
- Problem: To devise a walk through the city that would cross each of the seven bridges of Königsberg once and only once
  - Euler proved problem has no solution



Graph Problems – Hamiltonian Cycle

- Is there a simple cycle that connects all vertices in the graph
  - NP-Complete

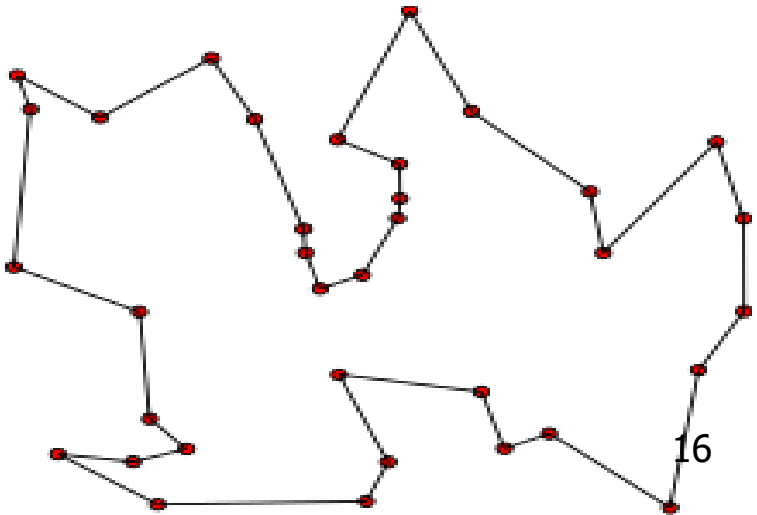
- Knight's Tour of chessboard
  - A sequence of knight's moves
  - Visit every square of a chessboard precisely once
  - Returns to its initial square



Graph Problems – Traveling Salesman

- A salesman wishes to
  - Visit a number of towns, and then
  - Return to his starting town
- Given the travelling times between towns, how should the travel be planned, so that:
  - He visits each town exactly once, and
  - He travels in as short time as possible

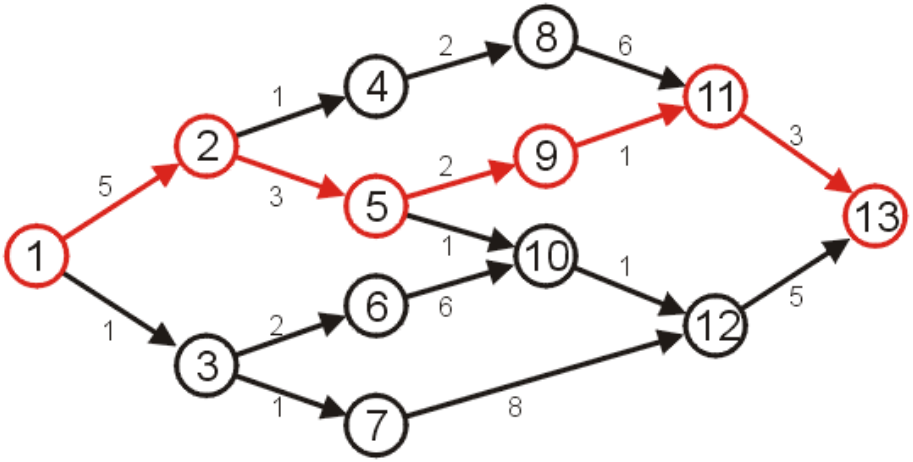
- Problem: Given a weighted graph G, provide shortest cycle that contains all vertices in G
  - NP-Hard problem

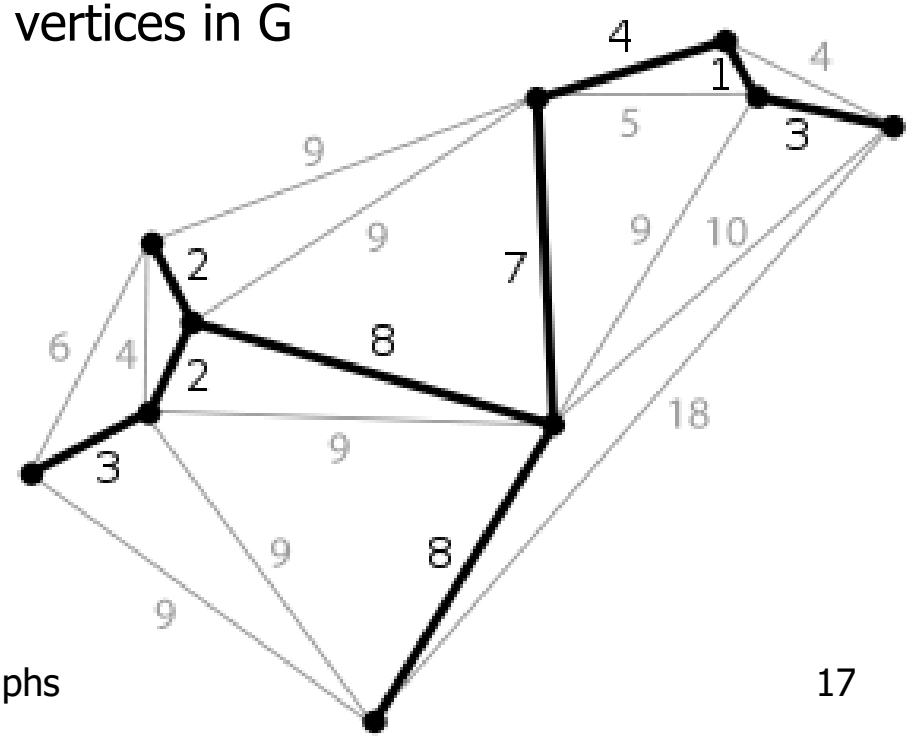


Graph Problems – Others

- Minimum-cost spanning tree
  - Given a weighted graph G, determine a spanning tree with minimum total edge cost

- Single-source shortest path
  - Given a weighted graph G and a source vertex v in G, determine the shortest paths from v to all other vertices in G





Any Question So Far?